以蛻變測試與突變分析檢驗語音助理的自然語言意圖路由

摘要

我自己寫的語音助理 J.A.R.V.I.S. 會先用一層本機的「意圖路由」判斷使用者講的話是要開程式、播音樂、控制家電，還是該交給大型語言模型。這一層（`router.py`）只有六十幾條正規表達式，但它的輸入是自然語言，同一件事有無限種講法，而且沒有一個標準答案可以拿來比對。本研究想回答的是：這種沒有標準答案的元件，測試要從哪裡來、這些測試可不可靠、留多少才夠、時間不夠時先跑哪些。我用蛻變測試來產生測試，用突變分析來評估測試，把這四個問題在自己的路由器上完整做了一輪，再把找到的錯修回去。

具體做法是定義五條蛻變關係（同義改寫、加語助詞、換語序、加無關前綴、近似句），分別用模板規則和 Gemini 模型（溫度 0、0.3、0.7、1.0）產生改寫句；另外針對正規表達式自己寫了七種突變運算子，產生 927 個突變體，建立一張 3,355 條測試 × 927 個突變體的矩陣，後面的縮減、排序、跨版本比較都從這張矩陣算出來。

主要結果如下。第一次蛻變測試就在 928 句裡找到 433 個違反，全部是手寫測試沒抓到的；經過五輪修正，模板批次的違反從 706 降到 7，模型批次從約 415 降到約 77，沒有反彈。生成器的溫度對找到的錯數、錯的集合和殺傷力都看不出影響，不同溫度之間的差異和同一個溫度重跑兩次的差異一樣大（句子 Jaccard 0.23–0.26 對 0.22–0.24），把批次放大四倍（n=20，兩萬三千句）之後仍然如此；真正有影響的是批次大小本身，去重後的錯從 110 增加到 273。以突變體當需求做縮減，3,355 條測試縮到 196 條（5.8%）仍保留全部殺傷力，同樣大小的隨機子集只剩 54%；additional 排序的 APFD 為 0.985，跑前 10% 的測試就找到全部的錯。在舊版本上縮減的子集套到新版本仍保留 84–85% 殺傷力（隨機 50%）；分析突變體之間的包含關係後發現 485 個可殺突變體裡只有 238 個（49%）是獨立的錯，一般的突變分數把隨機子集的能力高估了一倍。把真實使用紀錄當種子找到 4 個現行的錯，其中 3 個是缺規則，這種錯突變測試量不到。同一套方法搬到第二個受測對象（Home Assistant 的 520 條 zh-TW 意圖模板）上，縮減與排序的結論全部重現，但兩個受測對象錯的方向相反；第三個受測對象改用 Gemini 當路由器，沒有任何調校，在自然改寫上就比修了五輪的正規表達式穩（一致性違反 2.7% 對 7.4%），在模板句上則輸（3.9% 對 0.5%）；對 prompt 做突變發現七成的例句和定義拿掉也不影響結果（拿掉例句不掉分，拿掉定義一致性違反才翻倍），換成大一號的模型，錯的仍是同一批句子。縮減與排序的結論在三個受測對象、兩個模型、三種 prompt 寫法下都成立。人工標記 400 句改寫句，模型改寫有 96.5% 保留原意圖（n=20 另抽 400 句：94.8%），模板改寫 91.5%。

另外有一個原本沒預期到的發現：測試工具本身五次被自己的方法抓到錯（測試基座的標籤比對、種子含觸發詞、模型語意漂移、分析腳本的去重順序、模型輸出被截斷），每一次都會改變結論。「測試工具自己也要測」變成這份研究最實際的心得之一。

壹、前言

一、研究動機

J.A.R.V.I.S. 是我自己寫的語音助理。對著麥克風講一句話，它會開程式、播音樂、調音量、控制家電，或者把話交給大型語言模型回答。因為呼叫模型要錢也要時間，我在前面放了一層「本機路由」：先判斷這句話是不是簡單指令，是的話直接做，不是才送模型。這層路由大概是六十條正規表達式和關鍵字規則，每改一次都有對應的手寫測試，CI 一直是綠燈。

但真的用起來卻接連出事。「不要打開記事本」真的打開了記事本；「打開記事本好嗎」去找一個叫「記事本好嗎」的程式；「冷氣溫度調到 26 度」被天氣規則接走，回了一段天氣預報。三件事都是 CI 綠燈、上線後才發現。想了一下，原因其實很清楚：輸入是自然語言，同一件事有無限種講法，手寫測試只能寫我想得到的那幾句，而我想得到的講法跟路由器會處理的講法是同一個人想的，當然測不出問題。

後來讀軟體測試的文獻，發現這剛好對應到四個很老的問題：該測什麼（測試從哪裡來）、測試多可靠（測試本身會不會錯）、留多少（哪些測試是多餘的）、先測哪些（時間不夠時先跑哪些）。本研究用自己的產品當案例，把這四個問題完整做一輪，並把找到的錯修回產品。

二、研究目的與研究問題

編號	研究問題	怎麼回答
RQ1	蛻變關係自動生成的測試，能否找到手寫測試沒抓到的路由錯誤？模型生成與規則生成誰找得多、找得不一樣？	兩種生成器 × 五條關係，數違反、去重後的錯、人工標記改寫句的有效率
RQ2	生成參數（溫度）與模型非確定性對測試品質的影響	四個溫度 + 同溫度重跑三次，量句子重異、錯集合重異、殺傷力
RQ3	這些測試裡多少是多餘的？哪種縮減策略保留最多找錯能力？	自寫突變器 → 測試 × 突變體矩陣 → greedy / HGS / irreplaceable / 隨機
RQ4	只能跑前 k% 測試時，哪種排序最早找到錯？	同一矩陣：total / additional / 關係優先 / 隨機，APFD
RQ5	找到的錯修回去，路由變好多少？會不會修出新的錯？	修正前後五個版本重跑全部批次
RQ6	在舊版縮減 / 排序的測試集，程式改版後還有用嗎？	在 v4 的矩陣上決定，套到 v5 的矩陣上評估（反向也算）
RQ7	966 個突變體裡多少是獨立的錯？哪些運算子多餘？	突變體的包含關係、dominator 集、最小突變分數
RQ8	自己寫的種子跟真實使用差多少？真實指令找得到種子找不到的錯嗎？	使用紀錄裡的 41 條真實指令當第三批種子，比違反、比突變殺傷

編號	研究問題	怎麼回答
RQ9	以上結論是不是只對這一個 router 成立？	同一套關係、生成器、突變、縮減、排序跑在第二個受測對象（Home Assistant 的 zh-TW 意圖模板）上
RQ10	用模型當路由器，同一套測試會看到什麼？跟 regex 路由比，穩定度與成本差在哪？	Gemini 當 router（prompt 列 25 個意圖、回 JSON），同一套關係 / 縮減 / 排序；突變對象換成 prompt

RQ1、RQ2、RQ5、RQ8 是「測試從哪裡來」這一邊的問題，RQ3、RQ4、RQ6、RQ7、RQ9、RQ10 是「測試留多少、先跑哪些」這一邊；後者是本研究花最多力氣的部分。

貳、文獻探討

這一章只整理跟本研究直接有關的四條線，最後一節把 2025–2026 年最接近的幾篇工作和本研究放在一起比較。

一、蛻變測試（Metamorphic Testing）

Chen、Cheung 與 Yiu（1998）提出：當程式的正確輸出無法直接判定（所謂的 oracle 問題）時，可以改成檢查「多個輸入之間輸出應該滿足的關係」。例如 sin(x) 算不出標準答案，但 sin(x) 與 sin(π − x) 必須相等。對意圖路由來說，「同一句話換個講法應該走同一條路」就是一條蛻變關係，不需要知道正確答案是什麼。近年對自然語言處理模型的測試大多建立在這個想法上：Ribeiro 等人（2020）的 CheckList 把「加無關前綴、換同義詞、語意不變則輸出不變」寫成對 NLP 模型的行為測試。本研究的 R1 到 R4 就是這一類的「不變性」關係，R5 則對應他們所說的「方向性期望」：意思改了，輸出就應該跟著變。

二、突變測試（Mutation Testing）

DeMillo、Lipton 與 Sayward（1978）提出用「故意把程式改壞」來評估測試集：每一個改壞的版本叫突變體，測試集能讓多少突變體失敗（殺掉），就是突變分數。Jia 與 Harman（2011）的總覽整理了運算子怎麼設計、等價突變體（改了但行為沒變）的問題，以及成本。我一開始試了通用工具 mutmut，但它只改運算子和常數，碰不到正規表達式字串裡的分支，而路由器的邏輯幾乎全在字串裡，所以後來自己寫運算子。

三、測試集縮減與排序

縮減（reduction）是在保留「需求覆蓋」的前提下挑出最小的子集，本質上是集合覆蓋問題，沒有多項式時間的精確解，常用的近似法有 greedy 和 Harrold、Gupta 與 Soffa（1993）的 HGS。Lin 等人（2012、2014）提出先保留「不可取代的測試」（某個需求只有它能滿足）再補

齊其餘的做法，目的是降低回歸測試的成本；本研究把這三種策略和隨機對照都實作出來。排序（prioritization）是 Rothermel 等人（2001）提出的：測試全部都要跑，但把最可能找到錯的排前面，用 APFD（平均錯誤偵測百分比）衡量；total 和 additional 是兩種最基本的排法。

四、用大型語言模型生成測試

最近兩年用大型語言模型產生測試輸入變成熱門題目（Wang 等人 2024 年的綜述整理了一百多篇，測試輸入生成是最大的一類）。它吸引人的地方是句型多樣，風險則有兩個：語意漂移（改寫之後意思變了）和非確定性（同樣的參數重跑結果不同）。本研究把這兩個風險分別用人工標記（E1）和重跑底線（E2）量化，並且用模板生成器當對照組。

五、近期相關研究（2025–2026）與本研究的定位

「用改寫句當測試」在 2025–2026 年已經是常見的做法，本研究並不是第一個這樣做的。我認為自己的差別在於把後面三個問題（留多少、先測哪些、換了版本還有沒有用）用突變測試接上去，而且在三種實作、兩個模型、三種 prompt 寫法上重現同一組結論。下表是 2026 年 10 月查到的相關工作，標 arXiv 的是預印本，還沒有經過同儕審查。

文獻	他們做什麼	跟本研究的關係
Cho, Ruberto, Terragni（ICSME 2025）[10]；工具 LLMORPH（ASE 2025 demo）[11]	整理文獻裡 191 條 NLP 蛻變關係、實作 36 條，對三個 LLM 跑 56 萬次測試，平均違反率 18%；人工看 937 個違反，各關係的假陽性 0–70%；不處理非確定性	他們的假陽性問題就是 E1（模型改寫 3.5–5% 無效、模板 8.5%）；他們沒做的非確定性底線就是 E2。受測對象不同（路由器 vs LLM 本身），但 E11 把 LLM 也當受測對象測了一次
Guglielmi 等人（Empirical Software Engineering, 2026）[12]	用 GPT-4 對 40 個 Alexa skill 的指令生改寫句（79.5% 等價、37.6% 揭露 bug）；把改寫句補進 interaction model 能修掉大部分，但有的修不掉、有的新壞；GPT-oss-20B 便宜且差不多	最接近的工作。差別：他們測 Alexa 的黑盒 NLU，本研究測三種路由並加上突變、縮減、排序；他們的「補進去會新壞」對應五輪回饋的回歸集抓到 3 句；他們「生到飽和」對應 n=5 → n=20
Haroon, Khan, Gulzar（arXiv 2603.23443, 2026-03）[13]	突變驅動框架、8 個 LLM、22,374 個程式變體：語意保持的改動下 LLM 生的測試通過率掉到 79%，模型丟掉很多舊測試重生	同一個問題換個角度：E7 問舊版決定的測試集到新版還有用嗎（隔一輪 95%、隔四輪 41%），他們問 LLM 重生的測試跟不跟得上改版

文獻	他們做什麼	跟本研究的關係
Commey（arXiv 2601.22025, 2026-01）[14]	加「通用規則」到 prompt 不一定變好（Qwen 2.5 的 RAG 合規 26/30 → 9/30）；主張 prompt 改動要當回歸風險	E12 同方向：例句拿掉不掉分、定義拿掉才掉、全域規則對大模型幾乎沒作用；prompt 突變矩陣就是 prompt 的回歸測試
Liu, Meng（arXiv 2605.04665, 2026-05）[15]	900 句改寫基準：小模型遇到等價改寫，78% 的回答離開答案格式	本研究強制 JSON 輸出所以沒看到格式崩潰，但看到類似的現象：flash 的思考 token 把 JSON 截斷（51/187 解析失敗）
Yagubyan（arXiv 2605.28840, 2026-04）[16]	同一個工具呼叫任務跑兩次，選的工具、順序、參數一不一樣	單步版本就是本研究的「同句問兩次」：T=0 仍 86/87

查的時候沒有看到別人把 prompt 當成程式來做突變測試（E11、E12 的做法）。最接近的是 Commey 提出的「prompt 的改動要當成回歸風險」，但他沒有系統地刪掉 prompt 的每一段，看哪些部分其實是等價突變體。

※ 完整書目列在參考文獻。

參、研究方法



研究流程 · 蛻變測試（上兩列）與突變分析（下列）共用同一批改寫句與同一個 router

圖一：上半部用蛻變關係把種子句改寫、拿 router 跑、判定違反，違反歸因後修 router 再跑全部批次；下半部把 router 本身改壞成突變體，用同一批改寫句算「測試 × 突變體」矩陣，縮減、排序、包含關係、跨版本轉移都從這張矩陣算。

一、受測對象與測試基座

受測對象是 J.A.R.V.I.S. 的 `jarvis/router.py`：輸入一句中文，先做正規化（剝掉填充詞、客氣話、語助詞），再依序比對約 60 條規則，輸出是「目的地 + 目標」，例如 `open_application:記事本`、`youtube_play:晴天`、`home_control:冷氣=set=26`，或者「純問答」、「交給模型」。

為了能大量跑，我寫了 `harness.py` 把路由變成純函數：所有會碰裝置、打 API 的出口都換成記錄器，`route_label("幫我開一下記事本")` 直接回標籤，一句約 0.1 毫秒。

二、蛻變關係

關係	改寫方式	路由結果應該
R1 同義改寫	打開 → 開啟 / 啟動 / 幫我開	與種子相同
R2 語助詞、禮貌詞	幫我…一下、…好嗎、…謝謝	相同
R3 換語序	記事本幫我開一下	相同
R4 無關前綴	欸、那個、JARVIS	相同
R5 近似句	不要打開記事本、記事本打不開怎麼辦	**不同**（不該觸發）

種子句一開始有 41 條（涵蓋 29 種目的地），後來從突變分析反推補了 46 條，共 87 條。違反以「種子 × 關係 × 錯的目的地」去重之後稱為一個「錯」，免得「打開記事本好嗎」和「打開記事本啦」算成兩個。

三、兩種生成器

	規則式（對照組）	模型式
做法	模板：同義詞表、語助詞表、語序模板、近似句模板	Gemini 3.5 Flash-Lite，每條關係一段指令，輸出 JSON 陣列
變因	無	溫度 0、0.3、0.7、1.0；每個種子 × 關係生 5 句
成本	1 秒跑完	每批約 200 次 API 呼叫，有磁碟快取

	規則式（對照組）	模型式
過濾	長度 1–40 字；目標詞（程式名、歌名）必須還在句子裡	同左

量生成器用三個指標：有效率（過濾後留下的比例）、違反率（有效句裡違反關係的比例）、多樣性（字元 bigram 的 distinct-2）。

指標的定義如下。種子 s 的路由為 route(s)，關係 R 對 s 的一句改寫 t 算違反的條件是：

$$\mathrm{viol}(t) = \mathbf{1}[\,\mathrm{route}(t) \neq \mathrm{route}(s)\,] \;\; (R1\text{–}R4), \qquad \mathrm{viol}(t) = \mathbf{1}[\,\mathrm{route}(t) = \mathrm{route}(s)\,] \;\; (R5)$$

違反率只算有效句；去重後的「錯」以（種子, 關係, 實際路由到哪）為鍵；多樣性 dist-2 是不重複字元 bigram 佔全部 bigram 的比例；兩批測試的重疑用 Jaccard，句子集合與錯集合各算一次；人工標記的一致性用 Cohen's κ：

$$\mathrm{viol}\%_R = \frac{|\{t \in T_R :\ \mathrm{valid}(t) \wedge \mathrm{viol}(t)\}|}{|\{t \in T_R :\ \mathrm{valid}(t)\}|}, \qquad \mathrm{dist2} = \frac{|\mathrm{unique\ bigrams}|}{|\mathrm{bigrams}|}, \qquad J(A,B) = \frac{|A \cap B|}{|A \cup B|}, \qquad \kappa = \frac{p_o - p_e}{1 - p_e}$$

四、突變運算子

通用的突變工具碰不到正規表達式字串的內容，所以我自己寫了七種運算子，每一種模擬一種意圖路由真的會犯的錯：

運算子	例子	模擬的錯
ALT_DEL	`(?:打開|開啟|開)` → 少一個分支	少收一個同義詞
LOOK_DEL	`開(?!心|會)` → `開`	忘了排除誤觸
OPT_DEL	`(?:幫我)?` → `(?:幫我)`	前綴從可選變必要
KW_DEL	關鍵字 tuple 少一個	詞表漏字
NUM	`len(text) <= 6` → 5 / 7	邊界值錯
CMP	`<=` → `<`	邊界值錯
BOOL	and ↔ or、刪 not	條件寫反

現行的 router 產生 927 個突變體。「殺」的定義是：某條測試在原版路由到 X，在突變體路由到不是 X 的地方（或者直接爆掉）。測試來源是種子加上各批次裡原版通過的句子，共 3,355 條；矩陣分四段跑，每段約 2 分鐘。

五、縮減與排序策略

縮減的「需求」我試了兩種：突變體（貴但準）和「關係 × 種子」（便宜，每格留一條）。策略四種：greedy（每次挑殺最多尚未被殺突變體的測試）、HGS、irreplaceable-first（先留唯一能殺某個突變體的測試再補其餘）、隨機（同大小，30 次取平均）。排序也是四種：total（依殺傷數）、additional（每次挑殺最多尚未被殺的）、關係優先（R5 先）、隨機；指標是 APFD 和「跑前 k% 測試找到幾 % 的錯」。

設測試集 T、突變體 M（扣掉語法壞掉的），K(S) 為子集 S 殺掉的突變體，D 為 dominator 集（沒被更難殺的突變體包含的類）：

$$\mathrm{MS}(T)=\frac{|K(T)|}{|M|},\qquad \mathrm{retain}(S)=\frac{|K(S)|}{|K(T)|},\qquad \mathrm{MS}_{\min}(S)=\frac{|K(S)\cap D|}{|D|}$$

排序的 APFD：測試 n 條、錯（可殺突變體）m 個，TF_i 是第 i 個錯第一次被殺時的測試位置：

$$\mathrm{APFD}=1-\frac{TF_1+TF_2+\cdots+TF_m}{n\,m}+\frac{1}{2n}$$

跨版本轉移（E7）把 S 在版本 A 的矩陣上決定、在版本 B 的矩陣上算 retain 與 APFD；dominator-APFD 把 m 換成 |D|。

六、實驗設計

實驗	回答	做法
E1 有效性人工標記	RQ1 的信度	每個生成器 × 關係抽 40 句，共 400 句（n=20 六批另抽 400 句），打亂順序、看不出來源，標 1 = 保留原意圖 / 0 = 意圖變了或句子不通
E2 非確定性底線	RQ2	同參數（T=0）真的重打 API 兩次，與溫度之間的差異放在同一把尺上比
E3 突變測試	RQ3 的 ground truth	全批次矩陣；存活突變體抽 60 個人工分類，每個判斷附一句能殺掉它的句子並用程式驗證
E4 縮減	RQ3	兩種需求 × 四種策略
E5 排序	RQ4	四種排序、APFD 曲線
E6 版本回歸	RQ5	同一批測試跑在 router 的六個歷史版本（直接 `git show`）

實驗	回答	做法
E7 跨版本轉移	RQ6	v4、v5 各算一份矩陣；在 v4 縮減 / 排序，套到 v5 評估保留率與 APFD，同大小隨機對照
E8 突變體包含關係	RQ7	kill vector 相同的歸同一類，類間的子集包含找 dominator；用 dominator 算最小突變分數
E9 真實使用種子	RQ8	從 ~/.jarvis/profile.json 的使用紀錄匯出 41 條不重複指令（83 次使用），中文的 24 條當種子跑規則式生成器與突變矩陣
E10 第二受測對象	RQ9	Home Assistant zh-TW 意圖（41 意圖、520 條模板，hassil 整句匹配）：64 條模板展開種子 + 1,010 條抽樣句；模板突變運算子（刪分支 / 可選變必要 / 刪 skip 詞）2,748 個突變體；同一套 reduce / subsume
E11 第三受測對象	RQ10	gemini-3.5-flash-lite、T=0，prompt 每個意圖一行（名稱｜定義｜ target 格式｜例句）+ 5 條規則；87 條種子問兩次量一致率；規則式批次與模型批次各路由一次；違反分「對期望」與「對自己對種子的路由」兩種基準；prompt 突變 80 個（刪意圖 / 例句 / 定義 / 規則）× 187 句
E12 RQ10 的通則檢驗	RQ10	換 prompt 寫法（noex 沒例句、names 只剩名稱和格式）與換模型（gemini-3.5-flash）各動一次，重跑種子一致率、兩種批次的一致性違反；flash 再跑一次 prompt 突變矩陣與縮減 / 排序

全部工具都在專案的 `research/mt/` 目錄，每個實驗一個指令就能重跑；工具的煙霧測試和存活突變體的 killer 驗證都進了 CI。

肆、結果與討論

一、RQ1：蛻變測試找得到手寫測試沒抓到的錯

第一次用模板生成器產生 928 句，一秒就跑完，找到 433 個違反、去重後 83 個錯，全部是當時 CI 綠燈的手寫測試沒抓到的。

關係	生成	違反率	代表案例
R1 同義	48	14.6%	「開 github.com」走開程式而不是開網址
R2 語助詞	288	48.6%	「打開記事本好嗎」→ 目標變成「記事本好嗎」

關係	生成	違反率	代表案例
R3 換語序	52	96.2%	「記事本幫我打開」→ 交給模型
R4 前綴	288	41.0%	「欸 開 YouTube」→ 交給模型
R5 近似句	252	46.8%	「不要打開記事本」真的打開記事本

模型生成器找到的去重錯大約是模板的兩倍（第一輪修正後 110 對 16），而且兩邊互補：模板找到而模型沒找到的只剩 1 到 2 個。模型的優勢主要在 R3 換語序。模板只會產生「記事本幫我打開」這一種句型，模型會寫出「YouTube 這個 App 打開」「Chrome 幫我切過去」「26 度設定給冷氣」這種真的有人會講的句子。

我擔心生成句本身不可靠，所以做了人工標記（E1，400 句）：

生成器	保留原意圖（precision）	標 0 的主因
模型（四溫度合計 200 句）	96.5%	偷加動作（「今天很開心可以幫我記下來嗎」）、換動詞（播晴天 → 搜尋晴天）
規則式（200 句）	91.5%	模板硬套出不通的句子（「我靜音都打不開怎麼辦」「執行下載資料夾」）
規則式 R5 近似句	70%	同上，R5 模板最容易套出不通句

扣掉無效句之後，模型批次的違反率只從 13.0% 降到 11.4%，也就是說前幾輪找到的錯幾乎都是路由器的錯，不是生成器的錯；模板 R3 的 67.5% 違反也全部是真的。兩種生成器漂移的樣子不一樣：模型漂得少但比較隱蔽（偷加一個動作、換一個動詞），模板漂得明顯（句子根本不通），後者反而容易被濾掉。

後來把批次放大到 n=20（見 RQ2），我又抽了 400 句重標一次（六批 _n20 各關係抽 80，同一位標記者）：保留原意圖的比例是 94.8%，比 n=5 的 96.5% 低 1.7 個百分點，掉的全在 R3 語序（100% 降到 87.5%），因為每個種子後面 15 句的語序改寫開始不通（「去哪走走好放假」「26度把它設在冷氣」）；R2 標 0 的那句是模型多塞了一個動作（「麻煩幫我記錄今天過得挺開心的啦」）。扣掉無效句後違反率從 14.5% 降到 13.7%，也就是 n=20 多找到的錯有 94% 是真的（n=5 是 88%）。批次放大並沒有把漂移一起放大。

二、RQ2：溫度有沒有影響

六批都用同樣的 41 條種子（每批 964 句）：四個溫度各一批，再加上溫度 0 真的重打 API 兩次，當成「同樣設定重跑會差多少」的底線。

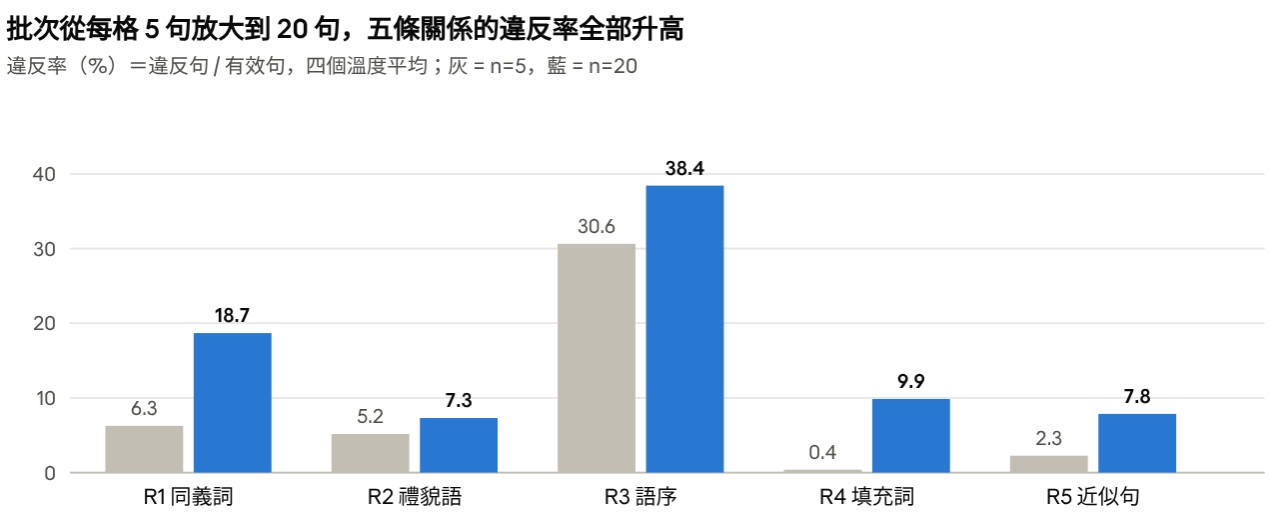
比較	句子 Jaccard	錯集合 Jaccard	共同的錯 / 只有一邊
溫度之間（6 對）	0.23–0.26	0.51–0.59	47–55 / 15–29
T=0 跑三次（3 對）	0.22–0.24	0.43–0.59	44–54 / 16–32
四個溫度全部共同的錯 / 聯集			38 / 116
T=0 三次全部共同的錯 / 聯集			39 / 115

從表裡可以看到三件事。首先，溫度 0 並不是確定性的：重跑之間只有 22–24% 的句子一樣。其次，溫度之間的每個數字都落在重跑雜訊的範圍內，「四個溫度共同的錯 38/116」和「溫度 0 跑三次共同的錯 39/115」幾乎一樣，所以「不同溫度會找到不同的錯」這個說法在這個規模下不成立。最後，殺傷力也一樣：四個溫度各殺 341、330、345、327 個突變體，溫度 0 的三次殺 341、336、331 個。

這裡我犯過一個分析上的錯。早期的結果顯示「T0 殺 329 個、T1 只殺 243 個」，看起來溫度有影響，後來才發現是測試去重的時候，同一句話只算給第一個批次，排在最後的 T1 被少算了一截；改成每句歸給所有含它的批次之後，差異就消失了。這件事讓我覺得，任何用模型生成測試的比較（溫度、prompt、模型）都應該附上一條重跑底線，不然看到的差異可能只是抽樣的運氣。

n=5 的結論有一個漏洞：也許只是批次太小才看不出溫度的影響。所以我把每個種子 × 關係的句數從 5 加到 20，同樣六批（41 條種子，router v5），共 23,000 句：

	n=5（v5 重算）	n=20
每批句子	964	3,808–3,846
每批違反（T0 / T0.3 / T0.7 / T1）	91 / 94 / 77 / 94	623 / 645 / 656 / 653
T=0 三次違反	91 / 96 / 95	623 / 659 / 681
每批去重錯（四溫度 / T=0 三次）	51–70 / 60–67	167–182 / 179–182
錯集合 Jaccard：溫度之間 / 重跑之間	0.34–0.48 / 0.37–0.51	0.56–0.59 / 0.56–0.58
句子 Jaccard：溫度之間 / 重跑之間	0.23–0.26 / 0.22–0.24	0.17–0.18 / 0.17–0.18
四溫度聯集的錯 / 六批聯集	110 / 127	273 / 315
dist2（R1 / R3）	0.41 / 0.46	0.21 / 0.27



run.py 六批 n=5 與 n=20 · 41 條種子 · router v5 · 每條關係取四個溫度的平均

圖二：R3 語序本來就是最難的（30% → 38%），放大後漲最多的是 R1 同義詞（6% → 19%）和 R4 填充詞（0.4% → 10%）。前 5 句用的是常見同義詞和「欸 / 那個」，後 15 句才輪到「叫出來 / 點開」和「那個… / 嗯～」。

溫度的結論沒有變：n=20 下四個溫度的錯是 167–182 個，溫度 0 自己重跑是 179–182 個；錯集合的 Jaccard 溫度之間 0.56–0.59、重跑之間 0.56–0.58。但批次大小本身有影響：違反率從 9% 跳到 17%，去重後的錯從 110 變成 273。我原本以為 n=5 找到的約 90 個違反已經接近路由器的上限，現在看來那只是「模型最先想到的 5 種講法」。多出來的錯我抽看了溫度 0 那批新出現的 246 個違反，大多是真的：填充詞沒收（「那個…」「嗯…」「Jarvis 幫我」「欸～」，整句掉到模型）；禮貌語黏進目標（「打開下載資料夾拜託你了嗎」變成 `open_folder:下載資料夾拜託你了`；語序句更糟，「把記事本打開來」變成 `open_application:來`、「播放歌手周杰倫的歌」變成 `youtube_play:手周杰倫`）；還有 R5 該擋沒擋的，整句包含種子片語的問句照樣觸發（「音量調大是要按哪個按鈕」「以圖搜圖怎麼用」「我剛剛不小心按到靜音了」、「取消前往 github.com」照開）。漂移是少數（「我想聽歌開心一點」路到播歌其實合理）。代價是多樣性砍半（dist2 從 0.41 掉到 0.21）、句子 Jaccard 從 0.24 掉到 0.17：後 15 句彼此很像，23,000 句裡只有 315 個不同的錯。違反的性質也變了，溫度 0 那批掉到模型的佔 55%（n=5 時 43%），放大批次多撈到的主要是「該觸發沒觸發」這種比較安全的錯。n=20 改寫句的人工標記結果在 RQ1 末尾。

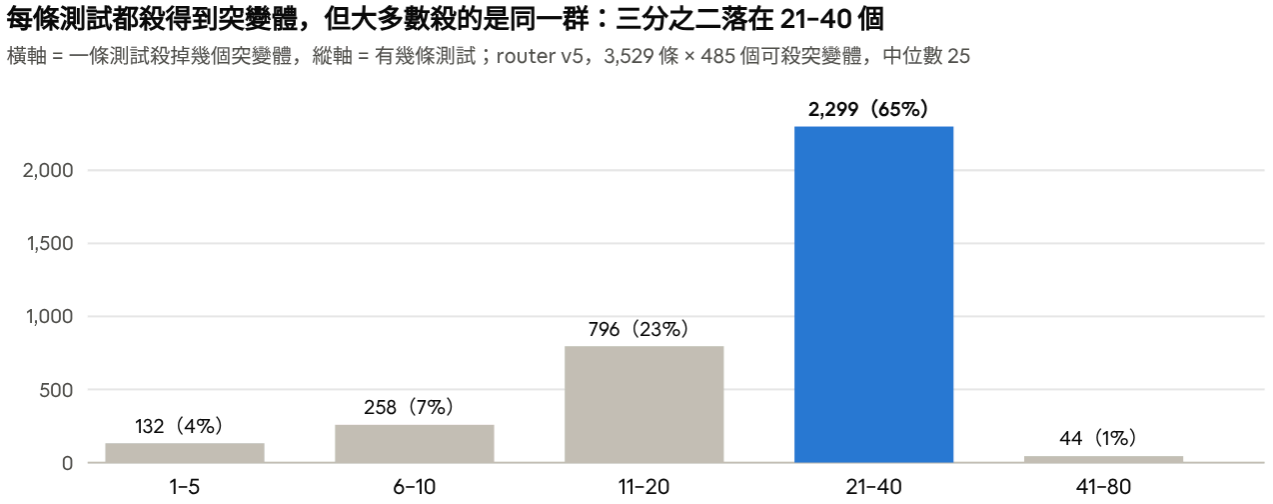
三、RQ3：測試可以縮到多小

主矩陣是 3,355 條測試 × 927 個突變體，殺掉 482 個，突變分數 0.520。

需求	策略	縮減後大小	保留的殺傷力	同大小隨機（30 次）
突變體	greedy	202（6.0%）	100%	54.2% ± 1.8
突變體	HGS	196（5.8%）	100%	53.7% ± 1.9

需求	策略	縮減後大小	保留的殺傷力	同大小隨機（30 次）
突變體	irreplaceable-first	196（5.8%）	100%	53.5% ± 1.7
關係 × 種子	greedy / irreplaceable	420（12.5%）	60.4%	66.5% ± 2.0
關係 × 種子	HGS	420（12.5%）	67.2%	66.7% ± 1.8

以突變體當需求時，三種策略幾乎一樣好：只留 6% 的測試就能殺掉全部原本殺得掉的突變體，而隨機挑同樣多只剩一半。改用「關係 × 種子」這種不用跑突變測試的便宜準則，縮減的結果就跟隨機沒有差別：「每格留一條」不知道哪一條有鑑別力。所以要縮減，就得先花時間算突變矩陣。



mutation.py 主矩陣（router v5）· 每條測試的殺傷數分組計數

圖三：沒有一條測試是完全沒用的（最少也殺 1 個），但分佈集中在 21–40 這一段，這些測試殺的是同一群「誰都殺得到」的突變體（正規化層、常用規則），所以留 6% 就能保留全部殺傷力；左邊只殺 1–5 個的那 132 條才是專門守某一條 regex 的。

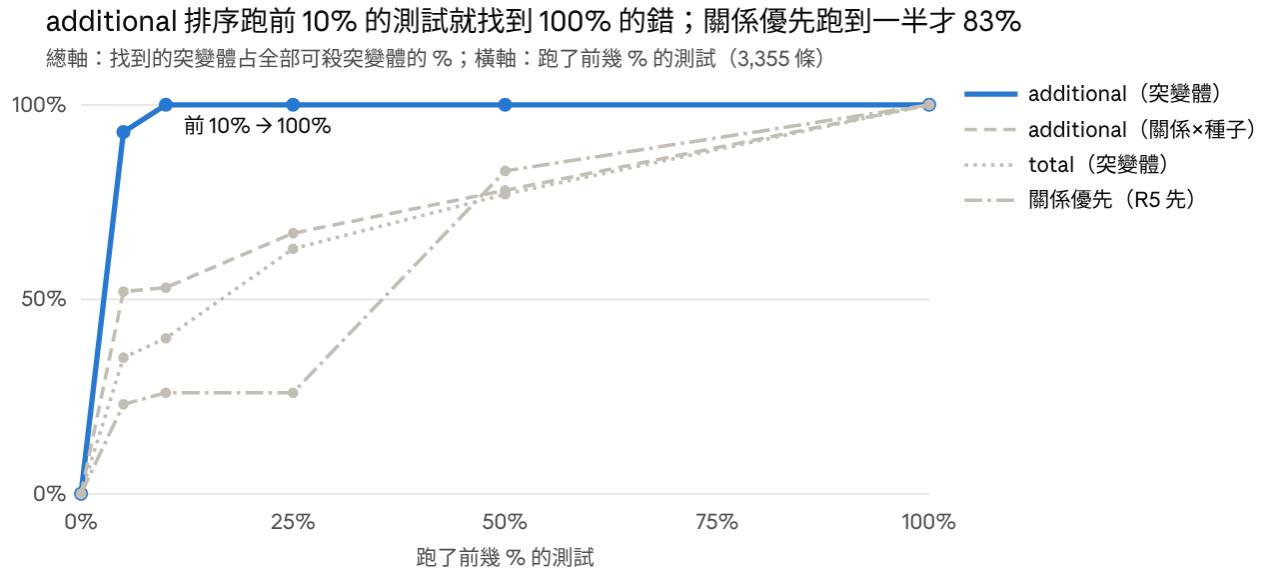
存活的突變體我抽了 60 個人工分類：沒有測試碰到的佔 77%、等價的 18%、測試基座看不到的 5%；扣掉等價突變體後，分數大約是 0.51。每一個「沒覆蓋」的判斷我都附一句應該能殺掉它的句子，再用程式驗證。第一輪的人工判斷 57 個裡錯了 19 個（33%），大多是那句話被另一條規則先接走了。把驗證過的 46 句補成種子之後，突變分數從 0.448 升到 0.516；這 46 句殺 191 個突變體，比原本 41 條種子的 156 個還多。但用模板再對這 46 句生 718 句改寫，只多殺 2 個：改寫換的是句型，突變體改的是詞表，兩者幾乎正交。

四、RQ4：先跑哪些測試

排序	APFD	跑前 5%	前 10%	前 25%	前 50%
additional（突變體）	**0.985**	93%	100%	100%	100%

排序	APFD	跑前 5%	前 10%	前 25%	前 50%
隨機（30 次）	0.836 ± 0.009				
additional（關係 × 種子）	0.783	52%	53%	67%	78%
total（關係 × 種子）	0.744	54%	55%	58%	75%
total（突變體）	0.729	35%	40%	63%	77%
關係優先（R5 先）	0.694	23%	26%	26%	83%

total 輸給隨機，這在文獻裡早就有人指出：殺最多的測試彼此重複，排在最前面的一百條都在殺同一群突變體。關係優先最差，原因是近似句找到的錯雖然最危險（該擋沒擋），但這些句子對正規表達式的可變點沒有施力，殺的突變體少。



reduce.py 在主矩陣（3,355 條 × 927 個突變體）的輸出 · 隨機排序 APFD 0.836，曲線略

圖四：曲線越早貼到 100% 越好，曲線下的面積就是 APFD。

五、RQ5：修了五輪之後

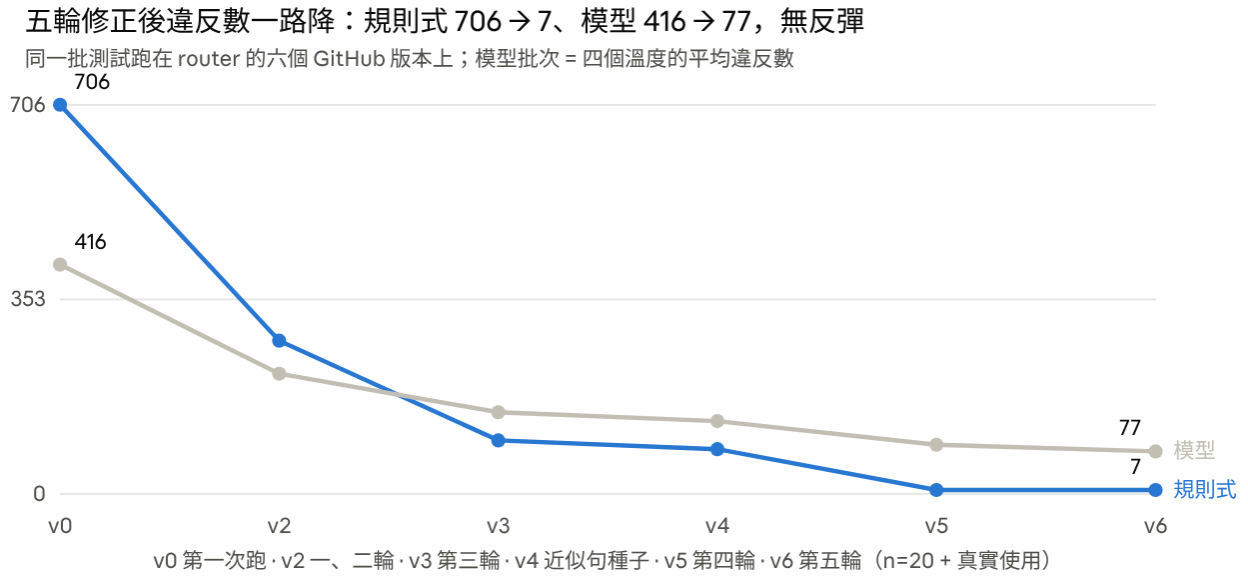
同一批 3,529 句測試跑在 router 的六個歷史版本上（都是 GitHub main 上的 commit；第一輪沒有單獨的 commit，跟第二輪一起推上去）：

版本	種子答對（41 / killer 46）	規則式 違反 / 錯	T0	T0.3	T0.7	T1	突變分數（殺 / 可用）
v0 `c060593` 第一次跑	39 / 23	706 / 172	413 / 164	420 / 170	423 / 165	407 / 165	0.468（175 / 374）
v2 `1352fae` 一、二輪	39 / 28	278 / 80	219 / 119	222 / 126	219 / 114	211 / 121	0.518（314 / 606）
v3 `0810b4d` 第三輪	39 / 45	97 / 35	148 / 78	147 / 76	147 / 72	149 / 84	0.513（444 / 865）

版本	種子答對（41 / killer 46）	規則式 違反 / 錯	T0	T0.3	T0.7	T1	突變分數（殺 / 可用）
v4 `7040c54` 近似句種子	41 / 46	81 / 32	131 / 70	133 / 69	128 / 69	134 / 79	0.516（441 / 854）
v5 `3f5e658` 第四輪	41 / 46	**7 / 5**	91 / 60	94 / 61	77 / 51	94 / 70	0.502（485 / 966）
v6 `db00e03` 第五輪	41 / 46	7 / 5	71 / 49	85 / 57	72 / 49	80 / 60	0.417（467 / 1,120）；加 n=20 六批 0.500（560）

從表裡可以看到幾件事。違反數一路下降，模板批次從 706 降到 7，模型批次從約 415 降到約 90，沒有反彈；中間其實出過狀況，第二輪曾經把報時規則修得太嚴，第四輪的第一版新壞了 3 句，但都被蛻變測試批次當成回歸集抓回來，在同一次修正裡解決。四個溫度在每一版的違反數都差不多（同一版內差距小於 6%），和 RQ2 一致。比較意外的是突變分數：v0 到 v5 都大約 0.5，但可用的突變體從 374 增加到 966。原因是 router 每修一輪就多收一批詞，每個新詞都是一個新的可變點，測試集也跟著長，比值才沒有掉；看絕對數，殺掉的突變體從 175 增加到 485。突變分數是一個比值，受測對象在長大的時候要看分子。

第五輪（v6）是把 n=20 六批的 315 個錯和真實使用紀錄的 4 個錯修回去：同一批句子的違反從 4,069 降到 2,919（修掉 1,150 個、零回歸），真實使用批次從 54 降到 3，模型 n=5 批次從約 90 降到 77。修的過程中三次被回歸集抓到：「到底」進了問句守門，害「剪貼簿裡到底有什麼」掉到模型；「囉」進了尾語表，害「哈囉」不再是招呼；「點開」進了開程式的觸發詞，害「點開心一點」變成開程式。這三個都在同一次修正裡解決。不過 v6 用原來的矩陣（n=5 六批）算的突變分數掉到 0.417，我一開始以為是修壞了，查了之後發現是第五輪加了 154 個可變點（填充詞、尾語、問句守門的詞表），而會殺這些新分支的句子在 n=20 批次裡，不在矩陣裡；殺掉的絕對數還少了 18 個，因為像「刪掉怎麼說」這種突變體被整句的「怎麼」守門蓋住，變成等價突變體。把 n=20 六批加進矩陣（13,879 條不重複句 × 1,120 個突變體），分數回到 0.500（殺 560 個）。規則變長了，測試集也要跟著長，分數才有意義。



versions.py 輸出與第四輪重跑結果 · 規則式 1,373 句，模型每批 964 句

圖五：模型批次在 v3 之後持平，到 v5 才再降，因為第四輪才處理 R3 語序，而模型批次有一半的違反都是語序；v6 再降到 77，剩下的是漂移和長尾。

六、RQ6：在舊版本上挑出來的測試，到新版本還有用嗎

RQ3 的「縮減後保留 100%」是在同一份矩陣上自己對自己，其實有點取巧。回歸測試真正的問題是：縮減在 v4 決定，程式改到 v5 之後，那份子集還殺不殺得到 v5 的錯？做法是兩個版本各自產突變體、各算一份矩陣（同一批 3,529 句測試），在 v4 的矩陣上縮減和排序，再套到 v5 的矩陣上評估。

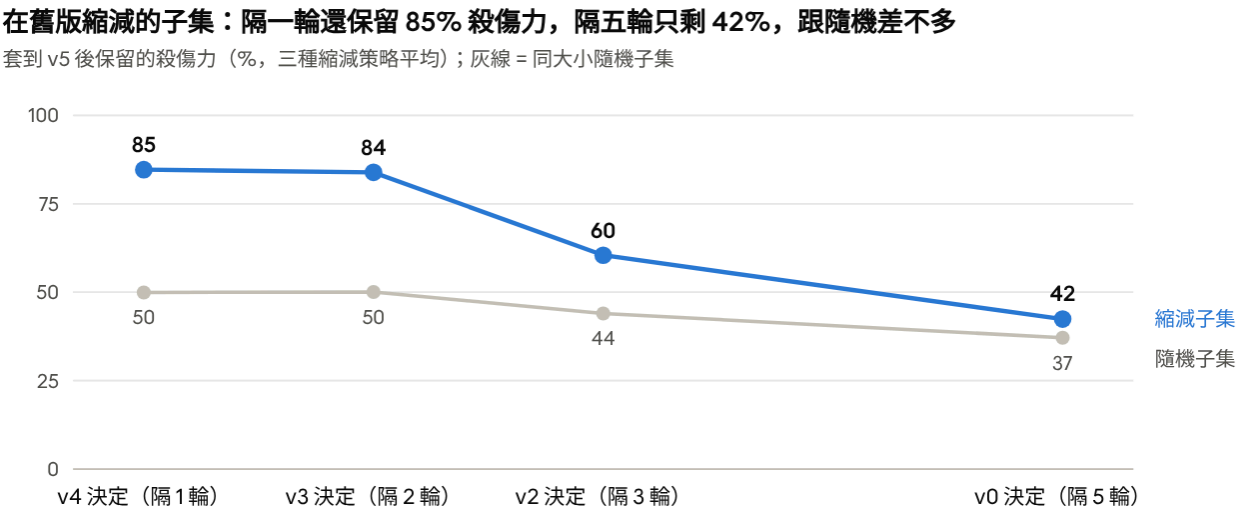
在 v4 決定 → 套到 v5	大小	在 v4 保留	在 v5 保留	同大小隨機（v5）
greedy	190	100%	**84.3%**	50.7% ± 2.0
HGS	183	100%	**85.2%**	49.4% ± 2.2
irreplaceable-first	184	100%	**84.5%**	49.7% ± 2.5

排序（在 v4 決定）	APFD@v4	APFD@v5
additional	0.986	**0.901**
total	0.730	0.760
關係優先	0.701	0.642
隨機	—	0.820 ± 0.009

反過來（v5 決定、套到 v4）三種縮減保留 89–90%（隨機 54%），additional 0.954。在 v5 掉的那 15% 全是 v5 新加的 regex（例如受詞在前的 `_OBJ_FIRST_RE`）的突變體：舊版本決定的子集不可能涵蓋新版本才出現的可變點，這是任何縮減方法的上限，不是策略的問題；三種策略轉移之後還是一樣好。實務上的意思是：縮減後的子集可以跨版本用，但 router 每加一條規則，就要把守它的 killer 種子補進去。

版本距離越遠掉越多（都套到 v5）：

在哪版決定	隔幾輪	縮減子集在 v5 保留	同大小隨機	additional APFD@v5	隨機 APFD
v4	1	84–85%	50%	0.901	0.82
v3	2	84%	50%	0.900	0.82
v2	3	60–61%	44%	0.797	0.82
v0	4	41–43%	37%	**0.716**	0.82



transfer.py · v4 / v3 / v2 / v0 各自縮減後套到 v5 · 隨機子集 30 次平均

圖六：隔一、兩輪幾乎不掉（v4、v3 都 84%），第三輪起急降。第三輪把鎖定、切換、家電的 regex 整個重寫，舊版選出來的句子對新 regex 的分支沒有鑑別力。

v3 到 v5 和 v4 到 v5 幾乎一樣，因為第四輪只加 regex 沒動舊的；v2 到 v5 掉到 61%，因為第三輪改寫了鎖定、切換、媒體鍵的規則，舊子集守的是已經不存在的可變點。隔四輪之後 additional 排序比隨機還差（0.716 對 0.82）：它把「在 v0 殺最多」的測試排前面，而那些測試守的規則在 v5 早就改掉了；total 反而穩（0.73–0.76），因為它本來就沒在利用版本特定的資訊。排序比縮減更怕過期。

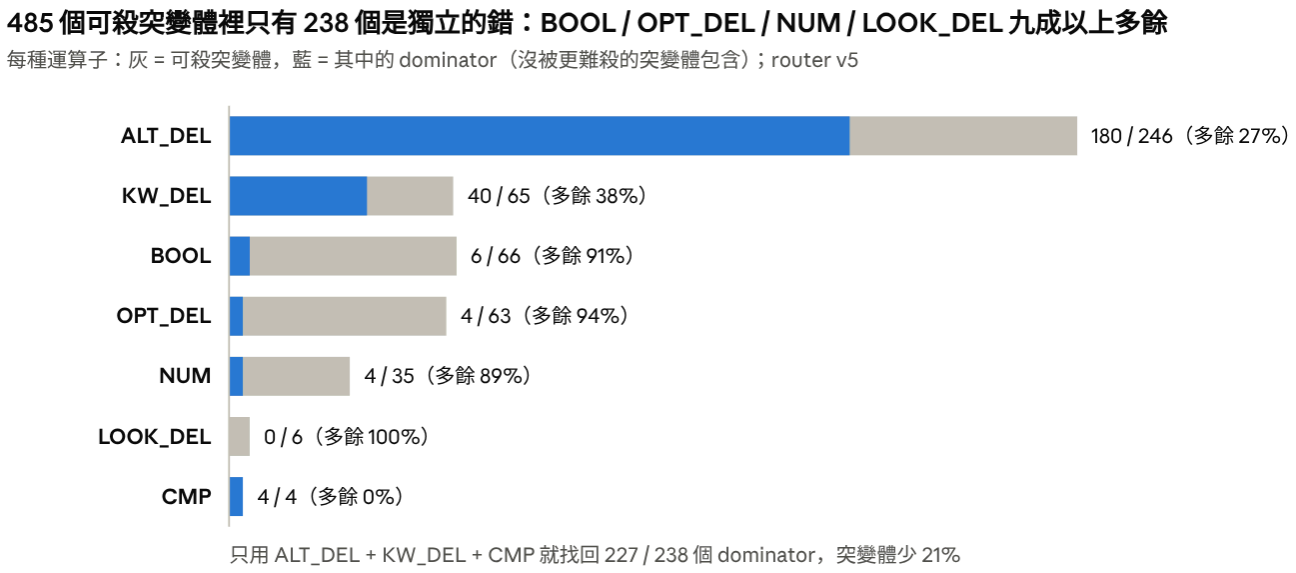
第五輪（v5 到 v6，加了 154 個可變點）我再算了一次，這次用看得到新分支的 13,879 句全集：在 v5 縮減的 272–275 條套到 v6 保留 94.5–95.5%（同大小隨機 43%），additional APFD 0.981（隨機 0.867）；反向 93%。隔一輪只掉 5%，比 v4 到 v5 的 15% 少，我的解釋是第四輪加的是整條新 regex，舊子集沒有句子碰得到；第五輪加的是既有 regex 的詞表，守舊詞表的句子有一部分順便碰到新詞表。另外，如果只用 n=5 的 3,529 句評估會看到 97%，那並不是更好，而是那些句子碰不到 v6 的新分支（可殺的突變體從 485 變 467）。轉移評估要用看得到新分支的測試集，不然新版本的洞根本不會被算進分母。

七、RQ7：966 個突變體裡有多少是獨立的錯

兩個突變體如果被同一群測試殺掉，對測試集來說就是同一個錯；如果殺 m 的測試都會殺 n，n 就是多餘的。沒有被更難殺的突變體包含的叫 dominator，用 dominator 算的最小突變分數不會被「同一個錯的好幾個突變體」墊高。在 v5 的矩陣上：485 個可殺突變體、413 個 kill vector 不同、238 個（49%）是 dominator。

運算子	可殺	dominator	多餘比例
ALT_DEL（刪 regex 分支）	246	180	27%

運算子	可殺	dominator	多餘比例
KW_DEL（關鍵字表少一個）	65	40	38%
CMP	4	4	0%
BOOL	66	6	**91%**
OPT_DEL	63	4	**94%**
NUM	35	4	**89%**
LOOK_DEL	6	0	**100%**



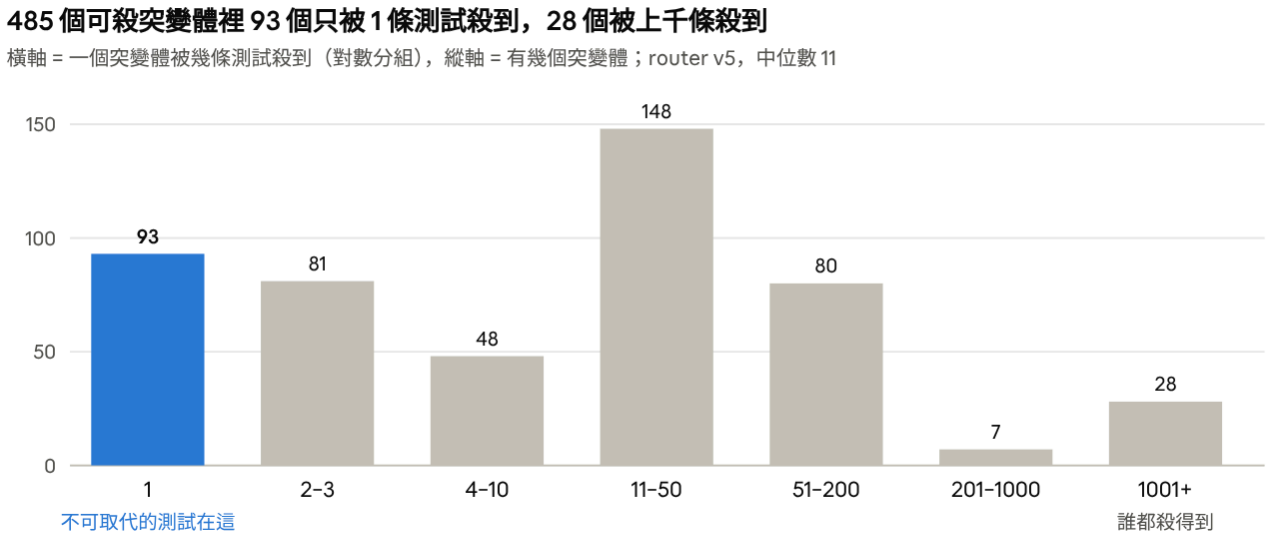
subsume.py · router v5 的 966 個突變體 × 3,529 條測試

圖七：改詞表的突變體（ALT_DEL、KW_DEL）才是獨立的錯；改運算子、邊界、可選群組的幾乎都被詞表突變體順便殺掉。

測試子集	大小	一般分數	最小分數
greedy / HGS / irreplaceable	212–215	100%	100%
同大小隨機	212–215	52%	**27%**
additional 前 2%	70	68.7%	38.2%
additional 前 5%	176	92.0%	83.6%
additional 前 10%	352	100%	100%

這張表讓我重新看了 RQ3 和 RQ4 的數字。一般的突變分數把隨機子集高估了一倍：隨機子集在一般分數下看起來有 52%，用最小分數算只有 27%，縮減策略和隨機的差距比 RQ3 那張表還大。運算子也有輕重：改運算子、邊界、可選群組的突變體九成以上被改詞表的突變體包含，只

用 ALT_DEL、KW_DEL、CMP 三種就能得到幾乎一樣的 dominator 集，突變體少 44%。最難殺的 dominator 全是第四輪新 regex 的分支，每一個只被 1 條測試殺到，那 1 條就是 golden test 或 killer 種子，這些才是測試集裡真正不可取代的測試。



mutation.py 主矩陣（router v5）· 每個可殺突變體被幾條測試殺到，對數分組

圖八：兩頭重。左邊 93 + 81 個只被 1–3 條測試殺到，那幾條就是 golden test 或 killer 種子，縮減時一定要留；右邊 28 個被上千條殺到（改正規化層的突變體），每條測試都順便殺掉它們，也是一般突變分數被墊高的來源。

用 dominator 重看 RQ3 和 RQ4：只用 ALT_DEL、KW_DEL、CMP 產突變體（765 個、可殺 315）就找回 227 / 238 個 dominator，三種縮減子集 202–204 條仍保留 100%，同大小隨機的最小分數同樣是 27%。突變體少了五分之一、可殺少了三分之一，結論沒變。

排序	APFD（全部突變體）	APFD（dominator）
additional	0.984	0.972
隨機	0.824	0.715
total	0.772	0.685
關係優先	0.645	0.520

以 dominator 類當「錯」算 APFD，排名不變、差距拉大：additional 幾乎不掉，total 和隨機掉 0.09–0.11，關係優先掉到 0.52，因為它們排前面的測試殺的是同一群多餘的突變體。

在 v6 的全矩陣（13,879 句 × 1,120 個突變體）重算一次：560 個可殺裡有 307 個（55%）是 dominator，比 v5 的 49% 高。n=20 的句子把一些原本被同一群測試殺的突變體分開了（ALT_DEL 的多餘比例從 27% 降到 19%），但 OPT_DEL、BOOL、NUM、LOOK_DEL 九成多餘的結論沒變。測試多了四倍，縮減比例從 6% 掉到 1.8%（250–256 條保留 100%），同大小隨機的最小分數從 27% 掉到 18%：測試集越大，隨機抽樣越沒用，縮減越值得。additional 排

序的前 2%（277 句）就殺光 560 個，跟 greedy 子集幾乎同大小，排序的前段其實就是縮減子集。最難殺的 dominator 換成第五輪的詞表分支，各只有 1 條測試殺到：每一輪新加的規則都會變成下一輪最不可取代的測試。

八、RQ8：真實使用紀錄當種子

前面 87 條種子都是我寫的（41 條憑印象、46 條對著存活突變體補）。第三批種子從使用紀錄（`~/.jarvis/profile.json` 的 history）匯出：使用者實際講過的 41 條不重複指令，共 83 次使用。其中 11 條（27%）是英文（Open insta、Play rickroll on yt、Can u shut down the laptop），router 一律交給模型；中文的 24 條當種子。

真實指令（使用次數）	現行路由	應該	為什麼 87 條種子沒抓到
搜尋瑞克搖並播放（3）	google 搜尋「瑞克搖並播放」	youtube_play:瑞克搖	「搜尋 X 並播放」是手機頁的功能，router 沒有這條規則
下載Spotify於桌面	install_app:Spotify於桌面	install_app:Spotify	目標後面接地點，種子都沒有
開啟螢幕	open_application:螢幕	交給模型	「螢幕」不是程式；開程式規則對目標不設防
截取看看這是什麼	純問答	camera_search	「截取」這個動詞沒收

模板生成器對 24 條真實種子生 390 句，有 54 個違反，全部來自上面這 4 條種子本身就錯（其餘 20 條零違反）。突變殺傷方面，真實批次 336 句殺 147 個突變體，其中只有 3 個是現有 3,529 條殺不掉的：「有哪些裝置」（使用次數第二高的指令，之前零覆蓋）、「框選」、「天氣」三個關鍵字。

這裡有兩個我覺得重要的發現。一是突變測試量不到「缺的規則」：4 個真實的錯裡有 3 個是 router 根本沒有那條規則，突變體只能改現有的規則，產生不出「缺一條規則」這種錯，所以突變分數 0.52 不代表路由器只錯一半，這類缺口只有真實使用才量得到。二是種子偏誤是真的，但有限：我自己寫的種子的殺傷覆蓋了真實使用的 98%（147 個裡的 144 個），但真實使用有 27% 是英文，而且最常用的指令之一完全沒測到。覆蓋率高只是因為種子跟規則是同一個人寫的。

九、RQ9：換一個別人寫的路由器

前面所有數字都來自同一個 router，而且種子、規則、突變運算子都是我一個人寫的，很難說結論不是只對它成立。第二個受測對象我選 Home Assistant 的 zh-TW 意圖模板：正式產品在用、別人寫的、繁體中文、41 個意圖 520 條模板。它跟 router.py 一樣是「規則 → 意圖」，但實

作方式相反：router 是用正規表示式在句子裡找關鍵字，hassil 是模板要整句精確匹配（只靠 skip_words 忽略「請、請問、謝謝、幫我、告訴我」）。模板語法 (a|b)、[可選]、<rule> 跟我的突變運算子一一對應，所以同一套關係、生成器、縮減、排序、包含關係分析可以原封不動跑上去（`run.py --subject hass`、`hass_mutation.py`）。種子 64 條從模板自動展開，另外用 hassil 的抽樣器對每個意圖最多抽 40 句（1,010 句），當成「使用者會講的句子」的替身。

先看蛻變測試。模板生成器對 64 條種子生 985 句，有 596 個違反（60%）；同一個生成器對 router v5 只有 7 個。

關係	生成	違反	HA 怎麼錯	router 怎麼錯（v0 → v5）
R1 同義詞	13	7（54%）	「啟動 / 執行客廳燈」不收；另 3 句是我的生成器把「開始除草」改成「打開始除草」，生成器的錯	15% → 0
R2 禮貌語	320	260（81%）	句尾 一下 / 好嗎 / 可以嗎 / 喔 全部 → none；只有「謝謝」在 skip_words 裡過得了	49% → 0
R3 語序	12	9（75%）	「客廳燈，打開」→ none；「客廳燈幫我打開」→ HassGetState（問燈開了沒）：skip 掉「幫我」後剩「客廳燈打開」，撞到問狀態的模板	96% → 0
R4 填充詞	320	320（100%）	句首 欸 / 那個 / 嗯 一律 → none	41% → 0
R5 近似句	320	0	整句匹配，不在模板裡的句子都是 none，不會誤觸發	47%（「不要打開記事本」真的打開）→ 0

run.py 規則式生成器 · router v0 928 句（第一次跑）與 Home Assistant 985 句

圖九：router 五條關係都會錯，Home Assistant 只錯在「該觸發沒觸發」的四條。整句匹配的實作天生不會誤觸發，代價是講法稍有不同就聽不懂。

整句匹配的 HA 只會「該觸發沒觸發」（R2、R3、R4），永遠不會 R5；找關鍵字的 router 兩種都會，v0 的時候 R5「不該觸發也觸發」佔了一半。HA 的 R2、R4 違反是設計上的取捨（寧可不懂也不要亂做），但「客廳燈幫我打開 → 問狀態」是真的錯：使用者要開燈，它回答燈開了沒。

再看突變、縮減、排序和包含關係。模板突變體 2,748 個（ALT_DEL 1,031、OPT_DEL 1,634、RULE_ALT_DEL 73、RULE_OPT_DEL 4、SKIP_DEL 6），矩陣 1,454 條測試 × 2,748 個突變體跑 8 分鐘。

	HA	router v5（RQ3 / RQ4 / RQ7）
測試	1,454（64 種子 + 1,010 抽樣 + 380 規則式改寫）	3,529
突變體 / 可殺（分數）	2,748 / 400（0.146；只用 64 種子時 175，0.064）	927 / 485（0.52）
dominator	259（65%）	238（49%）
運算子多餘比例	OPT_DEL 51%、RULE_ALT_DEL 34%、ALT_DEL 11%	OPT_DEL 94%、BOOL 91%、ALT_DEL 27%
縮減（需求 = 突變體）	→ 186–187 條（12.9%）保留 100%；同大小隨機 43%	→ 212–215 條（6%）保留 100%；隨機 52%
同大小隨機的最小分數	43%	27%
縮減（需求 = 關係 × 種子）	→ 1,193 條（82%）保留 94%；隨機 96%	→ 222 條保留 56–60%；隨機 57%
APFD：additional / total / 隨機 / 關係優先	0.956 / 0.790 / 0.746 / 0.634	0.984 / 0.772 / 0.824 / 0.645
APFD（dominator）：additional / total / 隨機 / 關係優先	0.944 / 0.739 / 0.753 / 0.569	0.972 / 0.685 / 0.715 / 0.520
additional 前 10% 測試找到	89%（dominator 83%）	100%

對照之後有四點。排序和縮減的結論沒有變：additional 最好、關係優先最差，在 dominator 分數下差距更開；三種縮減策略一樣保留 100%，隨機只有 43%，「關係 × 種子每格留一條」還是等於隨機。唯一的差別是 total 在 HA 上小贏隨機（0.790 對 0.746），在 router 上輸。突變分數低是因為測試少，不是突變體壞：64 條種子只殺 6%，加 1,010 句抽樣後 15%，520 條模板的分支絕大多數沒有句子碰到，跟 RQ3 的教訓一樣，存活體列的是種子缺什麼。哪種突變體是獨立的錯取決於實作：router 的 OPT_DEL 94% 多餘（可選群組幾乎都被詞表突變體順便殺），HA 的 OPT_DEL 只有 51%、ALT_DEL 只有 11%，因為整句匹配下 [的] 變必要就讓「打開客廳燈」整句失敗，是獨立的錯；所以 RQ7「以後只跑 ALT_DEL、KW_DEL、CMP」的省時建議只對正規表示式類的受測對象成立。最後，R5 在 HA 上零違反、殺傷也幾乎沒有（320 句裡只有 17 句殺得到突變體）：辨識成 none 的句子，模板怎麼刪都還是 none；關係優先排序在 HA 上最差，就是因為它把這 320 句排前面。

限制：HA 的種子是從模板展開的，不是真實使用（RQ8 那種缺口量不到）；模型生成器沒跑；裝置清單是我固定的 9 個名稱。

十、RQ10：用模型當路由器

第三個受測對象是把意圖清單寫進 prompt（25 個意圖，每個一行「名稱｜定義｜target 格式｜例句」，再加 5 條全域規則），每句話交給 gemini-3.5-flash-lite（溫度 0）回 `{"kind", "target"}`，標籤空間跟 router.py 一樣。我沒有做任何調校，prompt 寫完就跑，想看一個完全沒修過的模型路由器在同一套測試下長什麼樣。

先看種子和非確定性。87 條種子答對 78（90%）：錯的 9 條裡 2 條是 prompt 漏列意圖（download_file、list_windows）、3 條有爭議（「開 YouTube」答 open_url:youtube.com、問快速鍵答 info 而不是 model）、4 條真的錯（「用電腦打開記事本」沒切裝置、「現在才要打開記事本」照開）。同一句問兩次有 86 / 87 一致，翻的是「播報新聞」在 info 和 model 之間跳。溫度 0 仍然不是確定性的，跟 RQ2 的生成器一樣。

違反要分兩種基準算。對「期望路由」算，模板批次有 114 個違反，但其中 81 個來自那 9 條種子本身就答錯，這等於把「準不準」和「穩不穩」混在一起。改成對「受測對象自己對種子的路由」算（`--baseline own`，只量一致性）：

批次	模型路由（對自己）	regex v6（同一批）
規則式 1,373 句	52 / 1,339（3.9%）：R2 21 / 430、R4 14 / 430、R5 7 / 330、R3 7 / 76、R1 3 / 73	7（0.5%）
模型 T0 n=5	25 / 941（2.7%）	71（7.4%）

沒有調校過的模型路由在自然改寫上比修了五輪的 regex 穩（2.7% 對 7.4%），在模板句上輸（3.9% 對 0.5%）。這要公平地說：regex 的 0.5% 是五輪對著這批句子修出來的，模型則一句都沒看過。模型的錯有兩型，一是把「可以嗎」當問句（「音量調大可以嗎」答 info），二是整句包含指令就做（「播晴天會怎樣」照播），後者跟 regex v5 以前的 R5 錯是同一型。最有意思的是「開 YouTube」：種子本身答 open_url，但「欸 開 YouTube」「YouTube 幫我開」「啟動 YouTube」全部答 open_application，也就是種子才是異類，蛻變關係兩個方向都抓得到。regex 不會出這種錯，同一條規則永遠給同一個答案。

再來是對 prompt 做突變。刪 regex 分支的對應物就是刪 prompt 的一行：INTENT_DEL（刪一個意圖）、EX_DEL（刪例句）、DEF_DEL（刪定義）、RULE_DEL（刪一條全域規則），80 個突變體 × 187 句（87 條種子加模板每關係抽 20 句）= 1,496 次 API、22 分鐘。

運算子	突變體	殺	dominator
INTENT_DEL 刪一個意圖	25（1 個沒測試碰到）	24 / 24	16
EX_DEL 刪例句	25	7	5
DEF_DEL 刪定義	25	6	5
RULE_DEL 刪一條規則	5	5	0

刪意圖一定被殺（等於刪功能），而且 date、math、lens、hide、weather 各只靠 1 句殺到；但七成的例句和定義拿掉之後模型照樣答對，它從名稱就知道 open_application 是什麼，這些是 prompt 裡的等價突變體，有用的定義只在 info、model、set_volume 這種邊界。刪規則殺得到，但每條只被 3–7 句殺、全部被 INTENT_DEL 包含：規則對模型是加分，不是開關。突變分數 0.525、dominator 62%；縮減 187 → 25（13.4%）保留 100%（同大小隨機 43%）；APFD additional 0.955、total 0.886、隨機 0.741、關係優先 0.740；dominator-APFD additional 0.936、隨機 0.652。三個受測對象（regex、整句匹配、模型）的縮減與排序結論一樣，不一樣的是「什麼叫一個獨立的錯」：regex 是一個分支、模板是一個可選群組、prompt 是一個意圖。

成本是真正的取捨：模型路由每句上線都要一次呼叫（每句約 0.9 秒），突變矩陣 80 × 187 要 1,496 次 API；regex 的 966 × 3,529 離線 2 分鐘就跑完。這一節的限制：只試一個溫度（模型和 prompt 寫法各換了一次，見下面的 E12）；突變矩陣的測試只有 187 句；改意圖 X 的突變體只重跑原版路由到 X 的句子（跟 RQ9 同一個近似）。

上面只有一個模型（gemini-3.5-flash-lite）和一種 prompt 寫法，很難說「例句七成等價」是通則還是剛好。E12 把兩個變因各動一次：prompt 改成沒例句（noex）、只剩名稱和輸出格式（names）；模型換成同家族大一號的 gemini-3.5-flash。看三件事有沒有變：種子準確率和一致率、一致性違反率（`--baseline own`）、「例句 / 定義是等價突變體」。

設定	種子答對	同句兩次一致	規則式批次一致性違反	模型批次一致性違反	prompt 突變 殺 / 80（INTENT / EX / DEF / RULE）	縮減 187 → 25 保留	APFD additional
flash-lite · full	78 / 87	86 / 87	52 / 1,339（3.9%）	25 / 941（2.7%）	42（24 / 7 / 6 / 5）	100%	0.955
flash-lite · noex	**80 / 87**	84 / 87	58 / 1,347（4.3%）	32 / 961（3.3%）	—	—	—
flash-lite · names	75 / 87	85 / 87	**83 / 1,376（6.0%）**	**51 / 963（5.3%）**	—	—	—
flash · full	77 / 87	86 / 87	88 / 3,590（2.5%，n=20）	33 / 961（3.4%）	36（24 / **3** / **4** / 5）	100%	0.951

結果可以分三點講。例句是多餘的：拿掉 25 條例句，種子反而多對 2 條（「開 YouTube」在 full 下被 `開啟github.com → open_url` 那條例句帶偏，noex 答 open_application），一致性違反只差 0.4–0.6 個百分點，這跟 EX_DEL 七成存活是同一件事從另一邊看。定義有用，而且用在邊界：連定義都拿掉，種子掉 5 條、一致性違反翻倍（3.9% → 6.0%、2.7% → 5.3%），掉的都是 info、model、set_volume 的邊界句（「台北 top 10 美食」跑去 Google、「音量太大我的耳朵痛」真的把音量調小）。換大一號的模型，錯的還是那幾句：flash 錯的 10 條裡 8 條跟 flash-lite 重疊，都是 prompt 的問題（漏列 download_file、list_windows，info 和 model 的邊界沒寫清楚），不是模型能力的問題；大模型對例句、定義、全域規則更不依賴（EX_DEL 存活 18 → 22、DEF_DEL 19 → 21、五條全域規則只剩 1 個 dominator），但刪意圖仍是 24 / 24 全殺。flash 對表面噪音免疫，對語意改寫就不是：模型批次的 33 個違反裡，禮貌語和填充詞兩條關係合計只錯 1 句，其餘都在同義詞和語序，其中 7 個又是「開 YouTube」種子是異類，14 個在 info / model 邊界（「開會中安靜一點」答靜音，比種子的期望還合理）。縮減保留 100%、APFD additional 約 0.95，在三個受測對象、兩個模型、三種寫法下一次都沒變。

這一步還抓到測試工具的第五個 bug。flash 會「思考」，思考的 token 算進輸出上限，JSON 被截成 `{"kind": "`，187 句裡 51 句解析失敗。失敗是隨機的，同一句在原 prompt 下失敗、在突變 prompt 下成功就被算成「殺」，所以那一輪的突變分數 0.575 是被墊高的，修好（關掉思考、放大上限、失敗不進快取）之後是 0.45。錯誤的標籤會讓突變分數變好看，這是突變測試自己不會告訴你的事。

十一、成本與效益

最後把 E1 到 E12 每個方法花什麼、換到什麼放在一起看。時間是實測（單線程）；API 次數從快取的 meta 數出來；費用是付費層的上限估算（gemini-3.5-flash-lite 標準價 $0.30 / 1M 輸入、$2.50 / 1M 輸出，官方價目；以 1 個中文字約 1 token 粗估，路由 prompt 約 2,000 token；免費層存在，實際可能是 $0）。gemini-3.5-flash 的價格在價目頁上沒找到，不估。

測試從哪裡來：

方法	成本	產出	效益（router v0 第一次 / v5）
手寫 golden tests	五輪人工累積 120 條	120 句	當回歸集；蛻變測試找到的 433 個違反它一個都沒抓到
規則式生成器	0 API、1 秒	928 → 1,373 句	v0：433 違反 / 83 個錯；v5：7；保留原意 91.5%
模型生成 n=5（一批）	205 次 API、約 3 分鐘、≈ $0.07	964 句	v0 T0.7：396 違反 / 150 個錯（規則式的 2 倍）；v5：約 90 / 60；保留原意 96.5%
模型生成 n=20（一批）	193 次 API（跟 n=5 一樣多）、約 5 分鐘、≈ $0.2	3,850 句	v5：623–681 違反 / 167–182 個錯；四批聯集 110 → 273、六批 315

方法	成本	產出	效益（router v0 第一次 / v5）
換溫度 / 重跑	每批同上	同上	錯集合 Jaccard 溫度之間 = 重跑之間：多跑一個溫度 = 多跑一次 T=0
真實使用紀錄當種子	0 API	24 種子 → 390 句	4 個現行錯，3 個是缺規則，87 條自寫種子完全沒碰到

留多少、先測哪些：

方法	成本	效益
regex 突變測試	離線：927 個突變體 20 秒；矩陣 2,433 × 927 不到 9 分鐘、13,879 × 1,120 約 12 分鐘	3,355 → 196 條（5.8%）保留 100%（隨機 54%）；APFD 0.985；存活體分析抓到 2 處死碼
不跑突變，按關係 × 種子留一條	免費	保留 56–60% = 隨機
模板突變（Home Assistant）	離線 8 分鐘	縮減 100% vs 隨機 43%
prompt 突變（模型路由）	1,496 次 API、22 分鐘（flash-lite）/ 12 分鐘（flash，4 線程）、≈ $1	縮減 187 → 25 保留 100%（隨機 43%）、APFD 0.95；指出哪些例句 / 定義多餘
跨版本沿用舊子集	0（不重算）	隔一輪保留 84–85%、隔四輪 41–43%：每版重算的 9 分鐘比丟 15% 殺傷力便宜

受測對象上線的代價：

路由器	每句成本	一致性違反（規則式 / 模型批次）	修一個錯要什麼
regex v6	0 API、1.5 ms	0.5% / 7.4%	改 regex、跑 1 秒回歸、0 元
gemini-3.5-flash-lite	1 次 API、約 0.9 秒、≈ $0.0007	3.9% / 2.7%	改 prompt、重跑約 2,500 句驗證（≈ $1.7、10 分鐘）；T=0 仍 1% 翻答
gemini-3.5-flash	1 次 API、約 1 秒、價格未確認	2.5% / 3.4%	同上；錯的句子跟 flash-lite 同一批，換模型不解決 prompt 的洞

我從這三張表得到的結論是：模型生成器最划算的參數是 n 不是溫度，同樣 193 次呼叫，n=20 多找 2.5 倍的錯，多跑一個溫度則跟重跑一次沒有分別；突變測試的九分鐘是縮減唯一有價值的輸入，沒有它縮減等於隨機，有它 5.8% 的測試就保留全部殺傷力；模型路由贏在自然改寫的穩定度，輸在每句 600 倍的延遲和每句都要付費，對桌面助理來說，regex 收常見句型、模型接長尾（也就是現在的 → model fallback）是成本上合理的分工。

伍、回饋到產品

一、五輪修正

每一輪都是先把違反歸納成「原因」再修，不針對個別句子；每一類留一句進 CI 的 golden tests（最後 80 條）。

輪	來源	原因	修法	效果
1	規則式批次	startswith 型規則對前綴零容忍；句尾語助詞黏進目標；否定句、問句照做；URL 規則吃中文	正規化層（剝填充詞 / 客氣話 / 語助詞）、否定與問句守門、網域只認 ASCII	規則式 433 → 52；R4 41% → 0%、R5 40% → 0%
2	模型批次的 X → model	真人會講的句型沒收（意願句、受詞在前、平台片語）；關鍵字規則太貪（「播放一下周杰倫」變媒體鍵）	加句型；媒體鍵 / 報時改整句比對	模型 T0.7 396 → 194
3	模型批次剩下的錯	同一件事一堆講法（鎖定 / 切換 / 下一首）；問法的外圍詞；搜尋動詞 + 問資訊	每類一條 regex；先剝問法外圍詞；搜尋 + 資訊內容走純問答	T0.7 194 → 111
4	存活突變體、E2 的 R2 違反、R3 長尾	死碼（lens 的「找來源」被搜尋的「找」先接）；「…好嗎」把任務變成問答；「記事本幫我打開」沒規則收	觸發順序調整；問答判斷先正規化；受詞在前的句型翻回動詞在前（有分隔才直接翻，沒分隔只當最後 fallback）	規則式 81 → 7；模型 約 130 → 約 90

輪	來源	原因	修法	效果
5	n=20 六批的 315 個錯、真實使用的 4 個錯	填充詞後面的省略號 / 波浪號（「那個…」「嗯～」「Jarvis 幫我！」）沒收；禮貌語黏進目標（「拜託你了嗎」「謝謝你」）；語序句的動詞尾（「打開來」「啟動起來」）讓目標只剩「來」；整句包含指令片語的問句 / 陳述句照觸發（「音量調大是要按哪個按鈕」）；「搜尋 X 並播放」「下載 X 於桌面」「開啟螢幕」「截取」沒規則	填充詞與尾語詞表擴充、受詞尾綴（程式 / 軟體 / 捷徑）剝掉、SOV 動詞加「打開來 / 啟動起來 / 點開 / 叫出來」、問句守門加「怎麼 / 哪個按鈕 / 不小心 / 上次」、否定詞加「取消」；五條新規則；27 條黃金案例	n=20 六批 4,069 → 2,919 違反（修掉 1,150、0 回歸）；真實使用 54 → 3；模型 n=5 約 90 → 77

第四輪的修法是看 T0.7 那一批的清單歸納出來的：T0.7 降了 40%、其他批次降 30%，也就是從一批歸納的修正有七成能推到另一批。對使用者的意思是，「記事本幫我打開」「把截圖整理到資料夾好嗎」這類句子從「花 token 交給模型」或「變成問答不做事」變成本機直接做。

二、測試工具自己也要測

研究過程中測試工具本身被自己的方法抓到五次錯，每一次都會改變結論：

1. 測試基座的標籤比對：`computer:key:nexttrack` 被拆成不同結構，「下一首」的 77% 脆弱度有一半是假象。
2. 種子含其他規則的觸發詞：「搜尋台北天氣」改寫後被天氣規則接走，那不是路由的錯。
3. 模型語意漂移：「播晴天」被改成「搜尋晴天」，違反是測試的錯，E1 量化為 3.5%。
4. 分析腳本的去重順序：造成「T0 殺最多、T1 最少」的假結論，差點進報告。
5. 模型輸出截斷：flash 的思考 token 把 JSON 截成半句，解析失敗被當成標籤，突變分數從 0.45 被墊高到 0.575；修兩次才好，第一次改了寫入快取、沒改讀快取。

另外，存活突變體的人工分類如果沒有「每個判斷附一句可以執行的 killer」，第一輪 33% 的誤判也會直接進報告。這些經驗的共同結論是：測試生成、測試基座、分析腳本都是程式，都會錯，都要有自己的檢查。

陸、研究限制與效度威脅

- 種子選擇：41 條原始種子是我挑的，偏向我知道 router 會做的事；46 條 killer 種子是對著存活突變體寫的，偏向 regex 的詞表。緩解：第三批種子從真實使用紀錄來（RQ8），找到 4 個缺規則的錯；但只有一位使用者的紀錄，而且英文指令沒當種子。

- 單一受測對象：緩解：RQ9 把同一套跑在 Home Assistant 的 zh-TW 模板上，縮減和排序的結論重現；RQ10 再加了模型路由，E12 換過一個模型（flash）和兩種 prompt 寫法，結論不變。但兩個模型同家族，非 Gemini 的模型沒試；HA 沒跑模型生成器，種子是模板展開的。
- 生成句的有效性：E1（n=5 與 n=20 各 400 句）目前只有一位標記者，還沒算 Cohen's κ；標記者同時是開發者。兩次標記的比例（96.5% / 94.8%）和掉分的關係（R3）一致，但那是自洽，不是信度。
- 等價突變體：只抽 60 個人工估（18%），推回全體有誤差。
- 批次規模與模型：溫度在 n=5 和 n=20 都看不出影響，但生成器只試了 gemini-3.5-flash-lite 一個模型和一種 prompt；n=20 的六批雖然進了 v6 的矩陣，RQ3、RQ4、RQ6 的主要數字仍是在 n=5 矩陣上算的。
- 我既是開發者也是實驗者：修 router 的時候知道測試長什麼樣。緩解：只針對分類後的原因修，不針對個別句子；golden tests 和生成測試分開；第四輪用「看一批、驗其他批」量化推廣性。

柒、結論與未來工作

一、結論

1. 該測什麼：蛻變關係讓沒有 oracle 的自然語言路由也能自動產生測試，第一次就在 928 句裡找到 433 個手寫測試沒抓到的違反。模型生成器找到的錯是模板的兩倍，改寫有效率 96.5%；模板生成器則是免費、確定、一秒跑完的對照組。
2. 測試多可靠：溫度在 n=5 和 n=20 兩個規模下對找錯數、錯集合、殺傷力都沒有可見的影響，溫度之間的差異和溫度 0 重跑的雜訊一樣大；真正有影響的生成參數是批次大小（n=5 到 20，去重錯 110 到 273，多出來的大多是真錯）。用模型生成測試的比較應該附重跑底線。
3. 留多少：以突變體為需求，3,355 條縮到 196 條（5.8%）仍保留 100% 殺傷力，greedy、HGS、irreplaceable 三種策略幾乎一樣；不跑突變測試的便宜準則跟隨機沒有差別。
4. 先測哪些：additional 排序 APFD 0.985，跑前 10% 就找到全部的錯；total 排序輸給隨機。
5. 跨版本還有用，但會過期：在 v4 縮減的子集套到 v5 保留 84–85% 殺傷力（隨機 50%），掉的那 15% 全是新版才出現的規則（v5 到 v6 只加詞表，保留 95%）；隔四輪（v0 到 v5）只剩 41–43%，additional 排序更比隨機還差。縮減子集可以跨版本用，但每加一條規則要補守它的種子，排序更要每版重算。
6. 突變分數要看最小分數：485 個可殺突變體裡只有 238 個是獨立的錯；BOOL、OPT_DEL、NUM、LOOK_DEL 九成以上被詞表突變體包含，隨機子集的真實找錯能力只有一般分數的一半。分數也要看測試集有沒有跟上規則：v6 用舊矩陣算掉到 0.42，加 n=20 批次後回到 0.50；測試多四倍後 dominator 升到 55%、縮減到 1.8% 仍保留 100%、隨機子集最小分數只剩 18%。

7. 突變測試量不到缺的規則：真實使用紀錄找到的 4 個現行錯裡 3 個是 router 沒有那條規則，突變體只能改現有規則；自寫種子覆蓋了真實使用殺傷的 98%，但使用次數第二高的指令完全沒測到，27% 的指令是英文。
8. 結論不只對這一個 router 成立：同一套跑在 Home Assistant 的 520 條 zh-TW 模板上，縮減保留 100% 對隨機 43%、additional 最好、關係優先最差、便宜準則等於隨機，四個結論全部重現；但哪種突變體多餘跟實作有關（整句匹配下可選群組是獨立的錯），而且兩個受測對象錯的方向相反：找關鍵字的誤觸發，整句匹配的該觸發沒觸發。
9. 模型路由是另一種取捨，不是替代品：沒調校過的 Gemini 路由在自然改寫上比 regex v6 穩（一致性違反 2.7% 對 7.4%）、在模板句上輸（3.9% 對 0.5%），會出 regex 不會出的錯（種子本身是異類）、溫度 0 仍有 1% 翻答，每句要一次 API。prompt 突變顯示刪意圖一定被殺、刪例句或定義七成等價；換寫法驗證了這一點（拿掉例句不掉分、拿掉定義一致性違反翻倍），換大一號的模型錯的仍是同一批句子、例句與定義更多餘。縮減與排序的結論在三個受測對象、兩個模型、三種 prompt 寫法上一樣。
10. 回饋：五輪修正把模板批次的違反從 706 降到 7、模型批次從約 415 降到約 77，沒有反彈；第五輪把 n=20 找到的錯修掉 1,150 個、零回歸；生成的測試批次同時成為回歸集，抓到修正過程中新壞的句子。
11. 測試工具自己也要測：測試基座、種子、生成器、分析腳本、模型輸出截斷各被抓到一次錯，每次都會改變結論；錯誤標籤會把突變分數墊高（0.575 對實際的 0.45）。
12. 成本效益：模型生成器該花在 n 不是溫度（同樣 193 次呼叫，n=20 多找 2.5 倍的錯）；突變測試的九分鐘是縮減唯一有價值的輸入；模型路由每句 0.9 秒、要付費，regex 1.5 ms、免費，regex 收常見句型、模型接長尾是合理的分工。

二、未來工作

- 找第二位標記者完成 E1（n=5 與 n=20 各 400 句），算 Cohen's κ。
- 英文指令也當種子；收更多使用者的紀錄。
- 在 Home Assistant 上跑模型生成器，把「客廳燈幫我打開 → 問狀態」回報給上游。
- 把 prompt 突變矩陣的測試從 187 句加到全批；試非 Gemini 家族的模型。
- 從 Cho 等人整理的 191 條 NLP 蛻變關係裡挑適合意圖路由的補進來（現在只有 5 條）。
- 模型路由加一條「格式」關係（Liu 與 Meng 的輸出格式崩潰）：改寫句下 JSON 是否仍合法、kind 是否仍在意圖清單內。
- 照 Guglielmi 等人的做法，模型生成器「生到沒有新句為止」而不是固定 n，看錯數是否在 n=20 之後飽和。

參考文獻

1. T. Y. Chen, S. C. Cheung, S. M. Yiu. Metamorphic testing: a new approach for generating next test cases. Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology, 1998.

2. R. A. DeMillo, R. J. Lipton, F. G. Sayward. Hints on test data selection: Help for the practicing programmer. IEEE Computer, 11(4): 34–41, 1978. doi:10.1109/C-M.1978.218136
3. Y. Jia, M. Harman. An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering, 37(5): 649–678, 2011. doi:10.1109/TSE.2010.62
4. M. J. Harrold, R. Gupta, M. L. Soffa. A methodology for controlling the size of a test suite. ACM Transactions on Software Engineering and Methodology, 2(3): 270–285, 1993. doi:10.1145/152388.152391
5. G. Rothermel, R. H. Untch, C. Chu, M. J. Harrold. Prioritizing test cases for regression testing. IEEE Transactions on Software Engineering, 27(10): 929–948, 2001. doi:10.1109/32.962562
6. C.-T. Lin, K.-W. Tang, G. M. Kapfhammer. Test suite reduction methods that decrease regression testing costs by identifying irreplaceable tests. Information and Software Technology, 56(10): 1322–1344, 2014. doi:10.1016/j.infsof.2014.04.013
7. C.-T. Lin, K.-W. Tang, J.-S. Wang, G. M. Kapfhammer. Reducing the cost of regression testing by identifying irreplaceable test cases. Proc. 6th International Conference on Genetic and Evolutionary Computing (ICGEC), pp. 257–260, 2012. doi:10.1109/ICGEC.2012.124
8. M. T. Ribeiro, T. Wu, C. Guestrin, S. Singh. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 4902–4912, 2020. doi:10.18653/v1/2020.acl-main.442
9. J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, Q. Wang. Software Testing With Large Language Models: Survey, Landscape, and Vision. IEEE Transactions on Software Engineering, 50(4): 911–936, 2024. doi:10.1109/TSE.2024.3368208
10. S. Cho, S. Ruberto, V. Terragni. Metamorphic Testing of Large Language Models for Natural Language Processing. IEEE International Conference on Software Maintenance and Evolution (ICSME), 2025. doi:10.1109/ICSME64153.2025.00025（arXiv:2511.02108）
11. S. Cho, S. Ruberto, V. Terragni. LLMORPH: Automated Metamorphic Testing of Large Language Models. ASE 2025 demonstration paper.
12. E. Guglielmi, A. Spina, G. Bavota, R. Oliveto, S. Scalabrino, et al. Using large language models to test voice user interfaces. Empirical Software Engineering, 31:132, 2026. doi:10.1007/s10664-026-10859-7
13. S. Haroon, M. T. Khan, M. A. Gulzar. Evaluating LLM-Based Test Generation Under Software Evolution. arXiv:2603.23443, 2026（預印本）.
14. D. Commey. When Generic Prompt Improvements Hurt: Evaluation-Driven Iteration for LLM Applications. arXiv:2601.22025, 2026（預印本）.
15. A. Liu, J. Meng. Paraphrase-Induced Output-Mode Collapse: When LLMs Break Character Under Semantically Equivalent Inputs. arXiv:2605.04665, 2026（預印本）.

16. A. Yagubyan. How Consistent Are LLM Agents? Measuring Behavioral Reproducibility in Multi-Step Tool-Calling Pipelines. arXiv:2605.28840, 2026（預印本）.

17. 本研究程式碼與數據：github.com/Venshen885361/jarvis-desktop，`research/` 目錄。

※ 第 2–9 條的卷期頁碼與 DOI 已依 Crossref / ACL Anthology 核對（2026-10-13）；第 1 條為技術報告，無卷期；第 10–16 條為 2025–2026 新作，依 arXiv / Springer / 作者頁面核對，標「預印本」者未經同儕審查。格式交稿前依學校規定調整。